NIRIKSHAK

IoT-Based Predictive Maintenance Platform

A Comprehensive Report on Architecture, Preprocessing Pipelines, Machine Learning Specifications, and Code Implementation.

PROJECT TYPE:	IIoT Predictive Maintenance
DATE GENERATED:	June 24, 2026
BACKEND ENGINE:	Flask & Socket.IO (Eventlet)
MACHINE LEARNING:	3-Stage Sequential Inference Pipeline
DOCUMENT VERSION:	1.0.0 (Production-Ready)

1. System Overview & Technology Stack

Nirikshak is an advanced Industrial Internet of Things (IIoT) predictive maintenance platform designed to continuously monitor the mechanical and electrical health of industrial motor drives. It ingests high-frequency telemetry variables, performs automated signal preprocessing, runs a multi-stage sequential Machine Learning inference pipeline, and projects real-time analytics to a high-fidelity glassmorphic web dashboard.

Core Technology Stack

The system architecture is engineered using reliable open-source frameworks and libraries capable of high-throughput real-time telemetry processing:

- **Operating System:** Platform Agnostic (Windows / Linux / macOS) for flexible local or cloud deployments.
- **Backend Framework:** Python 3.10+ with Flask 3.0.x and Eventlet to enable high-concurrency event loops.
- **Real-time Gateway:** Flask-SocketIO establishing low-latency WebSocket connections for immediate data push.
- **ML Core Libraries:** Scikit-Learn (v1.4.2) for classical machine learning models and Joblib for serialization.
- **Data Handling:** Pandas and NumPy for vectorization, feature scaling, and statistics calculations.
- **Message Broker:** MQTT (HiveMQ Cloud broker using secure TLS/SSL encryption) for standard telemetry ingestion.
- **Database Storage:** Firebase Realtime Database (HTTPS REST / SDK) for real-time cloud data synchronization.
- **Frontend Client:** HTML5, Vanilla CSS (Glassmorphism layout tokens), Chart.js (v4.4), and Socket.IO client-side JS.

Monitored Sensor Parameters

The system reads five physical variables from the edge sensors (e.g. ESP32 + DHT11 + ADXL345 + ACS712) to construct the motor drive state footprint:

- **Temperature (C):** Monitors operational heat. Spikes indicate bearing friction or electrical overload.
- **Vibration (g):** Captures acceleration forces. Excessive vibration points to mechanical misalignment or bearing faults.
- **Current (A):** Measures electrical current draw. Overcurrent flags winding short circuits or heavy physical load.
- **Voltage (V):** Tracks terminal voltage supply. Fluctuations can trigger electrical instability warnings.
- **RPM / Speed:** Indicates rotor rotation rate. Slip fluctuations or low speeds point to load mismatch or

broken rotor bars.

2. System Architecture & Ingestion Pipeline

Nirikshak employs a decoupled, event-driven system architecture designed to isolate ingestion bottlenecks from telemetry processing and frontend rendering. The pipeline comprises four structural blocks:

A. Telemetry Ingestion (Sensor to Broker)

Physical edge sensors (DHT11, ADXL345, ACS712) linked to an ESP32 microcontroller publish JSON-formatted readings to HiveMQ Cloud. For staging or testing environments, a CSV Data Simulator (e.g. firebase_simulator.py) streams historical logs at regular 2-second intervals, simulating live operations.

B. Ingestion Loop (firebase_subscriber.py)

A dedicated background polling thread continuously checks the Firebase RTDB source node. When a new log is detected, the subscriber intercepts the raw dictionary, saves a backup copy to a local CSV repository (data/firebase_sensor_data.csv) to guarantee offline audit logs, and routes the dict directly to the ML engine.

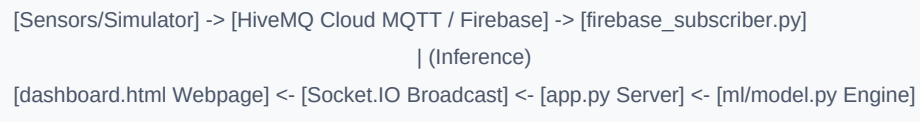
C. Inference & Routing (pipeline.py & ml/model.py)

The telemetry is passed through the 3-stage sequential ML models. The resulting enriched prediction payload is instantly routed in two directions: (1) it is written to the primary visualization database in Firebase under structured nodes, and (2) it updates the local state in app.py, triggering a WebSocket broadcast.

D. Client-Side Rendering (app.py to Browser)

Flask-SocketIO broadcasts the prediction payload ('sensor_update' event) to all connected dashboard sessions. The client UI intercepts this WebSocket message and updates the gauges, status indicators, and time-series charts dynamically within a 2-second cycle.

Data Pipeline Architecture Map:



3. Data Preprocessing & Feature Engineering

To ensure high stability and model inference accuracy, raw data inputs from physical sensors must be cleaned, imputed, and scaled before entering the machine learning models. The system runs the following pipeline defined in ml/model.py:

1. Parameter Alias Resolution

Different ingestion scripts may output slightly different naming conventions. The preprocessor resolves common aliases (e.g. mapping 'temp', 'Temperature(C)', and 'Temperature' to 'temperature') to normalize keys before subsequent data steps.

2. Missing Value Imputation (medians.pkl)

If a sensor fails or registers null values, the preprocessor loads a precalculated medians dictionary ('medians.pkl') compiled from the training dataset. This replaces the null with a typical healthy parameter, preventing runtime exceptions in the model pipeline.

3. Outlier Flagging (Flag-Not-Drop)

Rather than dropping records containing anomalies, the system flags outliers based on absolute thresholds. This allows the models to run on all rows while keeping the dashboard updated on limit violations:

- **Temperature Outlier:** Flagged if temperature exceeds 85.0 C.
- **Vibration Outlier:** Flagged if RMS vibration acceleration exceeds 4.0 g.
- **Current Outlier:** Flagged if current consumption exceeds 10.0 A.
- **Voltage Outlier:** Flagged if supply voltage deviates from the nominal 230.0 V by more than 0.5 V.
- **RPM Outlier:** Flagged if motor rotation speed exceeds 1450.0 RPM.

4. Standardization (scaler.pkl)

A pre-trained StandardScaler scales the imputed sensor values into a normal distribution with mean=0 and variance=1. This ensures that features with larger physical units (like RPM at 1450) do not dominate features with smaller physical units (like vibration at 0.8) during model calculations.

5. Rolling Features (utils/data_cleaner.py)

For historical batch analyses, the system uses a 5-step rolling window to generate the rolling mean and rolling standard deviation for temperature, vibration, and current. This captures time-series trend dynamics, enabling the models to recognize progressive failures.

4. The 3-Stage Machine Learning Engine

Nirikshak implements a sequential three-stage Machine Learning engine. Each model is chosen for its specific ability to address anomaly detection, categorical classification, or regression prediction:

Inference Stage	Model Selection	Primary Outputs	Design Rationale & Benefits
Stage 1: Anomaly Detection	Isolation Forest (isolation_forest.pkl)	Anomaly Status (Normal/Anomaly) Anomaly Score	Unsupervised algorithm that isolates anomalies in high-dimensional feature spaces by partitioning data. Highly efficient for flagging unexpected deviations from nominal footprints without requiring balanced labels.
Stage 2: Fault Classification	Random Forest Classifier (fault_classifier.pkl)	Fault Class (e.g., Bearing, Rotor) Confidence %	Ensemble classifier combining decision trees to capture non-linear relationships. Predicts exact failure category and provides probability scores to prevent false alarms.
Stage 3: RUL Estimation	Random Forest Regressor (rul_regressor.pkl)	Remaining Useful Life (RUL) in Hours	Combines normalized sensor variables and cumulative operating hours to project the remaining lifespan before breakdown. Crucial for scheduling preventive maintenance windows.

Robust Rule-Based Fallback System

To guarantee high system availability and fault tolerance, ml/model.py includes an embedded fallback logic (_rule_based_fallback). If the model pickle files are missing, deleted, or corrupted, the system catches the exception and falls back to a deterministic rule-based framework. This maps sensor anomalies (e.g. temperature > 75C, vibration > 4.5g) to predefined fault categories and estimates RUL using linear wear functions. The resulting payload features a 'model_source: rule-based-fallback' tag, keeping the UI dashboard running.

5. Codebase Structure & Core Modules

The Nirikshak codebase is organized into modular components. Below is the file structure and explanation of the core python scripts:

config.py

Central configuration module that loads environment variables from a local .env file. It contains definitions for Flask port numbers, database URLs, and the physical sensor thresholds (TEMP_MAX, CURRENT_MAX, etc.).

ml/model.py

The core ML execution engine. It lazy-loads all model files (scaler.pkl, isolation_forest.pkl, etc.) on demand. Includes the pre-processing logic, cumulative operating hours calculation (using datetime deltas), rule-based fallbacks, and the unified predict() entry point.

firebase_subscriber.py

A real-time database listener running in a background thread. It polls the Firebase RTDB ingest node, extracts incoming sensor parameters, executes the ML models, appends backups locally, and pushes predictions to the destination DB node.

utils/data_cleaner.py

A collection of data cleansing helper routines. Handles single-record range check validations and batch CSV operations such as Interquartile Range (IQR) outlier removal and rolling statistical feature calculations.

app.py

The Flask web server hosting application endpoints. It launches the background thread tasks, acts as the SocketIO websocket hub, serves static web pages, and implements a watchdog timer that injects simulated telemetry if live ingestion pauses for over 5 seconds.

templates/dashboard.html

The frontend dashboard utilizing a dark, modern glassmorphic interface. It binds to the websocket client, displaying live parameters across customizable dial gauges, rendering historical time-series charts, and hosting interactive diagnostic modals.

Visual Design and UI Glassmorphism

The frontend design uses high-contrast translucent cards, backdrop-filter blurs, and radial gradients. It features color-coded status elements: Green for Healthy operation (Normal), Yellow for warning levels, and Red for critical Anomaly status, giving operators instant machine status updates.

6. Execution Workflow & Operation

The operational lifecycle of a telemetry data point from the machine to the cloud and dashboard follows a structured pipeline. The sequence is detailed below:

- Step 1: Edge Acquisition: Edge sensors publish temperature, vibration, voltage, current, and RPM variables to the broker every 2 seconds.

- Step 2: Database Ingestion: The subscriber polls the source Firebase node and extracts the new reading payload.

- Step 3: Feature Engineering: The raw payload is imputed using 'medians.pkl', checked for outliers, and standardized using the scaler.

- Step 4: Model Inference: The scaled array passes through Stage 1 (Isolates anomalies), Stage 2 (Classifies fault type), and Stage 3 (Estimates RUL).

- Step 5: Logging and Sync: The output is appended to 'data/firebase_sensor_data.csv' and synchronized to the Firebase destination DB nodes.

- Step 6: Dashboard Update: Socket.IO emits the prediction data, and the web browser updates the interactive UI gauges and time-series charts instantly.

Interactive Diagnostic Modal System

To allow operators to perform deep diagnostics on individual sensors, the dashboard features a custom modal overlay. When clicking any sensor card, a detailed overlay displays: (1) historical sparklines plotting the last 50 parameters, (2) computed statistics (minimum, maximum, average) over the window, and (3) absolute physical limits. This aids in preventive maintenance decision-making.

Maintenance Action Recommendations

The system generates recommended diagnostic actions based on the classified fault type:
- Healthy: 'Continue normal operations. Next check in 250 operating hours.'
- Bearing Fault: 'Inspect bearing housing for wear or lack of lubrication. Schedule check within 24 hrs.'
- Misalignment: 'Re-align shaft coupling. Schedule check within 48 hrs.'
- Overheating: 'Check cooling fans and motor ventilation immediately. Reduce operating load.'
- Electrical Fault: 'Inspect stator winding, terminal voltage, and cables. Risk of short circuit!'
- Critical (<24 hrs RUL): 'CRITICAL ALERT: Machine failure imminent! Shut down and inspect immediately!'